Kotlin Deep Dive — Study Notes

> Advanced Android Course: From Code to Production
> Companion reference for the Kotlin Deep Dive session(s).
> All snippets are self-contained: copy-paste into a scratch file (`.kts` or a `main`) and they compile. Android-only examples are marked.

1. Type System & Language Core

1.1 Generics: Variance

Declaration-site variance — declared on the class:

```
open class Animal(val name: String)
class Cat(name: String) : Animal(name)

// `out` = covariant: T only appears in output positions (return types)
interface Producer<out T> {
    fun produce(): T
    // fun consume(item: T)  // ❌ compile error: T in `in` position
}

// `in` = contravariant: T only in input positions (parameters)
interface Consumer<in T> {
    fun consume(item: T)
}

class CatProducer : Producer<Cat> {
    override fun produce() = Cat("Tekir")
}

class AnimalConsumer : Consumer<Animal> {
    override fun consume(item: Animal) = println("feeding ${item.name}")
}

fun main() {
```

2. Functions & Functional Style

2.1 inline / noinline / crossinline — Bytecode Impact

```
inline fun measureNanos(block: () -> Unit): Long {
    val start = System.nanoTime()
    block()
    return System.nanoTime() - start
}

fun main() {
    val t = measureNanos { Thread.sleep(10) }
    println("took $t ns")   // no Function0 allocated — body copied into main()
}
```

What `inline` does:

- Copies the function body **and the lambda body** into the call site → zero `Function0` allocation, no virtual `invoke()`.
- Enables **non-local returns** (`return` inside the lambda returns from the enclosing

3. Classes, Initialization & Lifecycle

3.1 Initialization Order & Leaking `this`

Execution order: **superclass constructor → property initializers + `init` blocks in source order → constructor body.**

```
open class BaseScreen {
    open val size: Int = 10
    init { println("BaseScreen init, size = $size") }   // calls the OVERRIDDEN getter!
}

class DerivedScreen : BaseScreen() {
    override val size: Int = 20
}

fun main() {
    DerivedScreen()  // prints "BaseScreen init, size = 0"
                     // ← DerivedScreen.size backing field not yet initialized!
}
```

Rule: never call open members or read open properties in constructors/init blocks.

4. Equality, Immutability & State

4.1 == vs ===, equals/hashCode Contracts

```
data class Point(val x: Int)   // equals/hashCode generated from primary ctor properties

data class Profile(val id: Int) {
    var nickname: String = ""  // body property — IGNORED by equals/hashCode!
}

fun main() {
    val a = Point(1); val b = Point(1)
    println(a == b)    // true  — structural: a?.equals(b) ?: (b === null)
    println(a === b)   // false — referential: different objects

    // Boxed-int cache trap (-128..127):
    val x: Int? = 100; val y: Int? = 100
    val p: Int? = 1000; val q: Int? = 1000
    println(x === y)   // true  — cached Integer instances! never rely on this
    println(p === q)   // false

    // Body properties ignored:
    val u1 = Profile(1).apply { nickname = "A" }
    val u2 = Profile(1).apply { nickname = "B" }
    println(u1 == u2)  // true!

    // Arrays don't override equals:
    println(intArrayOf(1, 2) == intArrayOf(1, 2))             // false
```

5. Collections & Performance

5.1 Sequences vs Collections

```
fun main() {
    val list = (1..1_000_000).toList()

    // EAGER: 2 intermediate million-element lists, full passes
    val eager = list.filter { it % 2 == 0 }.map { it * 2 }.take(3)

    // LAZY: element-by-element pipeline, stops after 3 results, ZERO intermediate lists
    val lazy = list.asSequence()
        .filter { it % 2 == 0 }
        .map { it * 2 }
        .take(3)
        .toList()

    println(eager)   // [4, 8, 12]
    println(lazy)    // [4, 8, 12]
}
```

Sequences win: large collections + multiple ops, early termination (`take` , `first` , `any`), infinite sources (`generateSequence`).

6. Coroutines

> All snippets in this section need `kotlinx-coroutines-core`. Each is a complete runnable program.

6.1 Structured Concurrency, Scope, Job Hierarchy

Core idea: **every coroutine has a parent; a parent doesn't complete until all children complete; cancelling a parent cancels all children; a failed child cancels the parent (and siblings) by default.**

```
import kotlinx.coroutines.*

data class FullProfile(val user: String, val posts: List<String>)

suspend fun fetchUser(): String { delay(100); return "Ömer" }
suspend fun fetchPosts(): List<String> { delay(150); return listOf("p1", "p2") }

suspend fun loadProfile(): FullProfile = coroutineScope {   // parallel decomposition
    val user = async { fetchUser() }    // child 1
    val posts = async { fetchPosts() }  // child 2
```

7. Concurrency Beyond Coroutines

7.1 JMM in Kotlin Terms, @Volatile, Atomics

```
import java.util.concurrent.atomic.AtomicInteger
import kotlin.concurrent.thread

class Worker {
    @Volatile var running = true        // visibility: writes seen by all threads
    val processed = AtomicInteger(0)    // atomicity: increment is CAS, race-free

    fun loop() {
        while (running) { processed.incrementAndGet() }
    }
}

fun main() {
    val w = Worker()
    val t = thread { w.loop() }
    Thread.sleep(50)
    w.running = false       // without @Volatile, t might NEVER see this write
    t.join()
    println("processed ${w.processed.get()}")
```

8. Error Modeling

8.1 Result<T> & the runCatching Trap

```
import kotlinx.coroutines.*

suspend fun fetchData(): String { delay(1000); return "data" }

fun main() = runBlocking {
    val job = launch {
        // ⚠️ BUG: runCatching catches Throwable — INCLUDING CancellationException.
        // Cancellation is swallowed; the coroutine "completes" with a failure Result.
        val result = runCatching { fetchData() }
        println("swallowed cancellation, got: $result")   // this PRINTS — coroutine didn't cancel properly
    }
    delay(100)
    job.cancelAndJoin()
}
```

Fix — the cancellation-aware variant every codebase ends up writing:

```
import kotlinx.coroutines.*

inline fun <T> runCatchingCancellable(block: () -> T): Result<T> =
    try { Result.success(block()) }
```

9. Compiler, Interop & Metaprogramming

9.1 KAPT vs KSP

	KAPT	KSP
Mechanism	generates Java **stubs**, runs javac annotation processing	direct Kotlin compiler API, Kotlin symbol model
Speed	slow (stub generation ≈ an extra compile)	~2x faster typical
Kotlin awareness	"Java view" (no nullability, no suspend semantics)	real Kotlin types
Status	maintenance mode	the path forward (Room, Hilt, Moshi, Glide support it)

```
// build.gradle.kts — migration:
```

10. Modern Kotlin & Ecosystem

10.1 K2 Compiler (Kotlin 2.0+)

- Complete frontend rewrite (FIR): up to ~2x compile speed, one unified resolution structure.
- **Smarter smart casts** — stable across more scopes:

```
fun describe(x: Any): String {
    val isString = x is String
    return if (isString) "string of length ${x.length}"   // ✅ K2 smart-casts via a boolean variable
    else "not a string"
}

fun main() {
    println(describe("kotlin"))
    println(describe(42))
}
```

10.2 New Language Features (2.1+)

Guard conditions in `when` (requires `-Xwhen-guards` compiler flag on 2.1.x; stable in 2.2):

Dependency Cheat Sheet for the Snippets

Sections	Needs
1–5 (except 4.4 flow, 5.3 persistent)	stdlib only
4.4, 6.x, 7.2, 8.1	`org.jetbrains.kotlinx:kotlinx-coroutines-core:1.10.1`
5.3 persistent	`org.jetbrains.kotlinx:kotlinx-collections-immutable:0.3.8`
9.4	`org.jetbrains.kotlin:kotlin-reflect`
9.6 serialization, 10.2 previews, 10.5 detekt	plugin/flag noted inline, kept commented

Quick Self-Test (one question per block)

1. Why does reading an `open val` in a base-class `init` block print the default value (0/null)?
2. When does a non-capturing lambda allocate? A capturing one?
3. `lateinit` can't hold an `Int` — what's the bytecode-level reason?
4. Show how to mutate a `val cache: List<Item>` from outside the class — and two defenses.
5. For a 5-element list with `.filter.map`, which is faster: collection or sequence? Why?
6. `async` inside a normal scope throws — when does the parent find out: at `await()` or immediately?
7. Why can't you suspend inside a `synchronized` block?
8. What does `runCatching` break in structured concurrency?
9. Why does Gson need ProGuard keep rules but kotlinx.serialization doesn't?